

Entendendo os Decodificadores do Wazuh:

Processamento de Logs Sysmon

1. O que são Decodificadores?

No ecossistema do Wazuh, os decodificadores são componentes essenciais que transformam logs brutos e desestruturados em dados organizados. Como diferentes sistemas (Windows, Linux, Firewalls) geram logs em formatos distintos, o Wazuh utiliza decodificadores baseados em **Expressões Regulares (Regex)** para identificar o tipo de log e extrair campos específicos (como IDs de eventos, usuários, IPs e caminhos de processos).

2. Estrutura Técnica de um Decodificador

Um bloco de decodificação no arquivo XML do Wazuh (como o `windows_decoders.xml`) é composto por elementos que definem a lógica de captura:

- `<name>`: Identificador do decodificador.
 - `<parent>`: Define a hierarquia. O decodificador pai é verificado primeiro; se houver correspondência, o Wazuh prossegue para os decodificadores filhos.
 - `<prematch>`: Uma regex inicial de triagem. Se o log não contiver essa string, o decodificador é descartado rapidamente, economizando processamento.
 - `<regex>`: A expressão regular principal que contém grupos de captura — representados por parênteses `()`.
 - `<order>`: Define os nomes dos campos onde os dados capturados pela regex serão armazenados (ex: `id, sysmon.processId`).
-

3. Fluxo de Processamento: O Caso Sysmon

Ao receber um log do **Microsoft Sysmon**, o Wazuh executa um processo de três fases:

Fase 1: Pré-decodificação

O Wazuh extrai automaticamente informações básicas do cabeçalho do log, como:

- **Timestamp**: `Mar 29 13:36:36`

- **Hostname:** WinEvtLog
- **Program Name:** WinEvtLog

Fase 2: Decodificação (Onde a "Mágica" acontece)

O sistema compara o corpo do log com os decodificadores ativos. No exemplo do **Sysmon Event ID 1 (Process Create)**:

1. **Prematch:** O log contém a string `INFORMATION(1)` e termina com `Hashes?`
Sim.
2. **Extração de ID:** A regex `Microsoft-Windows-Sysmon/Operational:\s+\((\d+)\)` busca o número dentro dos parênteses após o nome da operação. No log exemplo, o valor **"1"** é extraído e salvo no campo `id`.
3. **Extração de Campos Detalhados:** Outro bloco decodificador usa grupos de captura para mapear o restante do log:
 - `ProcessGuid` → `sysmon.processGuid`
 - `Image` → `sysmon.image`
 - `CommandLine` → `sysmon.commandLine`

Fase 3: Comparação com Regras (Próximo Passo)

Uma vez que o log está decodificado e os campos estão preenchidos (conforme mostrado na saída do teste abaixo), o Wazuh passa esses dados para o **Mecanismo de Regras**, que decidirá se aquele evento deve gerar um alerta de segurança (ex: execução de um script PowerShell suspeito).

4. Resultado da Decodificação (Exemplo JSON)

Após o processamento bem-sucedido, o log bruto é transformado em um objeto estruturado que facilita a busca e a visualização no Dashboard:

```
"data": {
  "srcuser": "WIN-P57C9KN929H\\Alberto",
  "id": "1",
  "sysmon": {
    "processId": "3784",
    "image":
"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
    "commandLine": "powershell.exe -file test.ps1",
    "integrityLevel": "Medium"
```

```
}  
}
```

Essa estruturação permite que analistas de SOC criem filtros específicos, como "mostrar todos os processos onde `integrityLevel` é igual a `High`", algo impossível de fazer eficientemente com logs brutos.

Entendendo o Motor de Regras do Wazuh: Detecção de Ameaças

1. O que são as Regras do Wazuh?

Se os decodificadores são os "olhos" que extraem os dados, as **Regras** são o "cérebro" do Wazuh. Elas analisam os dados estruturados pelos decodificadores e aplicam condições lógicas para determinar se um evento é comum ou se representa uma ameaça à segurança. Quando uma correspondência é encontrada, um alerta é gerado com um nível de gravidade específico.

2. Anatomia de uma Regra (Análise Técnica)

Uma regra profissional, como a utilizada para detectar anomalias no **Sysmon**, possui campos fundamentais para o trabalho em um SOC:

- `id`: Identificador exclusivo (ex: 184666). Essencial para filtrar alertas e criar exceções.
- `level`: Gravidade de 0 a 15. Níveis altos (como o 12 no exemplo) geralmente disparam notificações por e-mail ou Slack.
- `<if_group>`: Condição de dependência. Esta regra só será avaliada se o evento já tiver sido classificado anteriormente no grupo `sysmon_event1`.
- `<field name="...">`: Onde a lógica de detecção reside. Ele compara o valor extraído pelo decodificador (ex: `sysmon.image`) com uma string ou regex (ex: `svchost.exe`).
- `<mitre>`: Mapeamento direto para a base de conhecimento **MITRE ATT&CK**, permitindo saber exatamente qual técnica o atacante está usando (ex: `T1055 - Process Injection`).

3. Investigação Prática: O Caso do `svchost.exe` Suspeito

No exemplo prático, observamos a transição de um evento informativo para um alerta crítico:

Fase 1: Evento Genérico (Regra 184665)

Quando o log do Sysmon indica apenas a criação de um processo comum (como o `powershell.exe`), a regra disparada é de nível baixo, servindo apenas para registro histórico.

Fase 2: Detecção de Anomalia (Regra 184666)

Ao alterarmos o log para simular o processo `C:\WINDOWS\system32\svchost.exe` sendo iniciado a partir do `explorer.exe` (ParentImage), o Wazuh identifica um padrão de **Evasão de Defesa**.

- **Por que é suspeito?** O `svchost.exe` é um processo crítico do sistema que normalmente não deve ser iniciado diretamente pelo usuário via Explorer. Atacantes frequentemente tentam "mascarar" malwares usando nomes de processos legítimos do Windows.
-

4. Resultado do Teste de Regras (Fase 3)

A saída do **Ruleset Test** do Wazuh fornece ao analista de SOC todas as informações necessárias para a resposta ao incidente:

```
**Phase 3: Completed filtering (rules).
  id: 184666
  level: 12
  description: Sysmon - Suspicious Process - svchost.exe
  mitre.id: {"id":["T1055"],"tactic":["Defense
Evasion"],"technique":["Process Injection"]}
  pci_dss: ["10.6.1","11.4"]
**Alert to be generated.
```

5. Raciocínio de SOC: Impacto e Conformidade

O uso de regras bem estruturadas permite que o SOC atenda a requisitos de conformidade internacionais automaticamente:

- **PCI DSS / HIPAA:** A regra já vem mapeada para controles específicos (ex: `pci_dss_11.4`), provando que o ambiente está sendo monitorado contra intrusões.
- **Triagem Automática:** Ao invés de olhar milhares de logs, o analista foca apenas no "Alerta gerado" de nível 12, reduzindo a fadiga de alertas e

aumentando a eficiência na detecção de **Process Injection** ou **Privilege Escalation**.

Hierarquia de Regras e Supressão no Wazuh

1. Sumário Executivo

Este módulo foca na lógica avançada de detecção do Wazuh, especificamente na utilização de **condições de precedência (if_sid, if_group)**. O objetivo é demonstrar como criar uma árvore de decisão onde uma regra depende do sucesso de outra, permitindo a identificação de comportamentos legítimos e a redução de ruído (falsos positivos) em processos críticos do sistema, como o `svchost.exe`.

2. Metodologia: A Árvore de Decisão do Analista

Diferente de regras isoladas, a detecção profissional em um SOC funciona em camadas. No Wazuh, as regras são processadas de cima para baixo em uma relação de **Pai e Filho**:

1. **Regra Base (Pai):** Identifica um evento genérico (ex: Criação de processo Sysmon).
2. **Regra de Anomalia (Filho):** Identifica um desvio suspeito (ex: `svchost.exe` iniciado por um usuário).
3. **Regra de Refinamento/Exceção (Neto):** Identifica se aquele desvio é, na verdade, uma operação legítima do sistema (ex: `svchost.exe` iniciado pelo `services.exe`).

3. Evidências Técnicas e Lógica de Condição

3.1. Implementação da Condição if_sid

A condição `if_sid` (If Signature ID) cria uma dependência direta. Uma regra só será avaliada se o ID da regra especificada tiver sido acionado anteriormente.

Exemplo de Regra de Exceção (ID 184667):

XML

```
<rule id="184667" level="0">
  <if_sid>184666</if_sid> <field name="sysmon.parentImage">\\services.exe</field>
  <description>Sysmon - Legitimate Parent Image - svchost.exe</description>
</rule>
```

3.2. Fluxo de Processamento de Log

Durante o teste com um log do Sysmon onde a `parentImage` era `C:\Windows\services.exe`, o motor de regras seguiu este fluxo invisível:

- **Passo 1:** O log corresponde à regra básica de evento Sysmon ID 1 (`if_group: sysmon_event1`).
- **Passo 2:** O log corresponde à regra de "Processo Suspeito" (ID 184666) porque a imagem é `svchost.exe`.
- **Passo 3:** O log corresponde à regra de "Imagem Pai Legítima" (ID 184667) porque o pai é o `services.exe`.

Resultado: A regra final disparada é a 184667 com **Nível 0**, o que significa que o SIEM entende que este evento é seguro e **não gera um alerta visual para o analista**, economizando tempo de triagem.

4. Raciocínio de SOC: Gestão de Falsos Positivos

No dia a dia de um SOC, o excesso de alertas (alert fatigue) é um dos maiores desafios. A técnica de `if_sid` demonstrada aqui é a base para:

- **Tuning de Regras:** Ajustar o SIEM para ignorar ferramentas administrativas conhecidas.
- **Contextualização:** Permitir que o sistema entenda que o mesmo processo (`svchost.exe`) pode ser malicioso se vier do `explorer.exe`, mas é legítimo se vier do `services.exe`.

5. Mapeamento MITRE ATT&CK

- **Tática: Defense Evasion (TA0005)**
- **Técnica: Masquerading (T1036)**
 - *Nota Investigativa:* Atacantes usam nomes de processos como `svchost.exe` para se misturar ao tráfego legítimo. O uso de hierarquia de regras (`if_sid`) permite ao SOC validar a árvore de processos (Process Tree) e detectar quando o "Pai" do processo não condiz com o comportamento padrão do Windows.

6. Conclusão Técnica

A capacidade de encadear regras (`if_sid` e `if_group`) transforma o Wazuh de um simples leitor de logs em um motor de correlação inteligente. Dominar esta hierarquia permite ao analista de segurança documentar exceções de forma granular, garantindo que o SOC foque apenas em ameaças reais, mantendo uma alta fidelidade nos alertas gerados.

Este é um excelente exemplo de como a **Engenharia de Detecção** funciona na prática dentro de um SOC. Quando as regras padrão são muito genéricas, o analista deve criar **Regras Locais (Custom Rules)** para adicionar contexto específico ao ambiente de negócio.

Aqui está o fechamento dessa documentação para o seu portfólio, focada em **Deteção de Ameaças em Linux (Auditd)** e **Redução de Falsos Positivos**.

Custom Rules no Wazuh: Monitoramento de Integridade em Linux (Auditd)

1. O Problema: Regras Abrangentes vs. Especificidade

Regras padrão (como a 80790) detectam criações de arquivos de forma genérica. No entanto, para um analista de SOC, criar um arquivo no diretório `/home` é diferente de criar um script no diretório `/tmp` ou `/var/log`. O uso de diretórios temporários é uma tática comum de atacantes para hospedar malwares voláteis.

2. Metodologia: Criando Deteções Contextuais

Para aprimorar a capacidade de resposta, implementamos uma regra personalizada no arquivo `local_rules.xml`. O objetivo é elevar a precisão do alerta quando comandos como `touch` são usados em locais sensíveis.

Fluxo de Ingestão do Auditd:

1. **Log Bruto (Auditd):** O kernel Linux registra a Syscall (ex: `syscall=2` para *open/create*).
 2. **Decodificador:** O Wazuh extrai campos como `audit.exe (/bin/touch)` e `audit.directory.name (/var/log/audit)`.
 3. **Regra Local (ID 100002):** Filtra especificamente se o diretório de trabalho (`audit.cwd`) contém palavras-chave como `tmp`, `temp` ou `downloads`.
-

3. Implementação Técnica (`local_rules.xml`)

A regra abaixo demonstra o uso de **Variáveis Dinâmicas** na descrição, o que facilita a leitura do alerta pelo analista de Nível 1.

XML

```
<group name="audit,">
  <rule id="100002" level="3">
    <if_sid>80790</if_sid> <field name="audit.cwd">downloads|tmp|temp</field>
    <description>Audit: $(audit.exe) created a file named $(audit.file.name) in
$(audit.directory.name).</description>
    <group>audit_watch_write,</group>
  </rule>
</group>
```

Validação via Ruleset Test:

Ao testar um log onde o arquivo `malware.py` é criado, o Wazuh agora identifica não apenas que um arquivo foi criado, mas gera uma descrição legível:

- **Resultado:** Audit: /bin/touch created a file named /var/log/audit/tmp_directory1/malware.py in /var/log/audit.
-

4. Raciocínio de SOC: Por que isso é importante?

Visibilidade de TTPs

Atacantes frequentemente realizam o download de ferramentas de pós-exploração para o diretório `/tmp` devido às permissões de escrita facilitadas. Ao criar essa regra, o SOC consegue:

- **Monitorar Drop de Arquivos:** Identificar quando binários são baixados em pastas temporárias.
- **Análise de Procedência:** Verificar se o executável que criou o arquivo (`audit.exe`) é um utilitário esperado ou um shell reverso.

Mapeamento MITRE ATT&CK

- **Tática:** Persistence (TA0003) / Defense Evasion (TA0005)
- **Técnica: T1105 - Ingress Tool Transfer:** O monitoramento de diretórios de download e temporários é a primeira linha de detecção contra a entrada de ferramentas maliciosas no host.

5. Conclusão

A personalização de regras no Wazuh permite que o SIEM se adapte às particularidades do ambiente Linux. Ao utilizar `if_sid` para estender regras existentes e capturar campos de Syscalls do `auditd`, o analista transforma um log de sistema obscuro em um alerta acionável e rico em contexto para a equipe de resposta a incidentes.

Implementação de Cadeia de Regras: Monitoramento de Diretórios Temporários (Wazuh)

1. Objetivo Técnico

Desenvolvi uma hierarquia de regras personalizadas no Wazuh para monitorar a criação de ficheiros em diretórios críticos (/tmp, /temp, downloads). O foco é detetar scripts e binários suspeitos, reduzindo o ruído através de exceções para ferramentas autorizadas.

2. Configuração de Regras (local_rules.xml)

Implementei a lógica de "funil" onde cada regra filho aumenta a severidade com base em extensões e nomes de ficheiros.

XML

```
<group name="audit,">
  <rule id="100002" level="3">
    <if_sid>80790</if_sid>
    <field name="audit.directory.name">downloads|tmp|temp</field>
    <description>Audit: Ficheiro criado em diretório temporário por
$(audit.exe)</description>
  </rule>

  <rule id="100003" level="12">
    <if_sid>100002</if_sid>
    <field name="audit.file.name">.py|.sh|.elf|.php</field>
    <description>ALERTA: Script/Binário criado em área temporária:
$(audit.file.name)</description>
    <mitre><id>T1105</id></mitre>
  </rule>

  <rule id="100006" level="0">
    <if_sid>100003</if_sid>
    <field name="audit.file.name">malware-checker.py</field>
    <description>Exceção: Ferramenta autorizada de Red Team.</description>
  </rule>
</group>
```

3. Validação e Resultados

Utilizei o **Ruleset Test** para validar a cadeia de eventos. O sistema identificou corretamente que, embora o ficheiro estivesse num diretório vigiado (Regra 100002) e tivesse uma extensão crítica (Regra 100003), ele pertencia a uma exceção aprovada, silenciando o alerta final.

Destaques da Análise:

- **Eficiência:** A regra 100003 foca na técnica **T1105 (Ingress Tool Transfer)** do MITRE.
- **Tuning:** O uso da Regra 100006 (Nível 0) evita a fadiga de alertas, garantindo que o SOC foque apenas em ameaças reais.

4. Conclusão

Esta estrutura permite-me ter uma visibilidade granular sobre a entrada de ferramentas no sistema Linux. Ao automatizar a distinção entre scripts legítimos e potenciais malwares através de `if_sid`, otimizoo o tempo de resposta a incidentes e a precisão do SIEM.